Two Dimensional Arrays

Department of Computer Science
University of Pretoria

27 May 2024

- An array is a contiguous list of elements in memory of the same type.
- int myArray[5]; defines 5 `int`-size contiguous memory locations.
- Values can be assigned to the reserved memory locations, either when (or after) the memory is (or has been) reserved.

```
int myArray[] =
        {1, 2, 3, 4, 5};
```

```
int myArray[5];
...
for (int i = 1; i <=5; i++) {
        myArray[i-1] = i;
}
```

- The first element in the array is indexed 0 and is accessed using the square brackets [] operator, myArray[0].
- What happens when the bounds of the array are exceeded, for example myArray[6]?
- What is the result of the following statement, cout << myArray << endl;?

- How can I calculate the number of elements in my array if the array is holding elements of a fixed size?
  `int myArraySize = sizeof(myArray) / sizeof(int);`
- When sending an array as a parameter to a function, the size of the array MUST be sent as a parameter as well, that is either
  `void printArray(int values[], int size);` or
  `void printArray(int *values, int size);`,
- You cannot calculate the size of an array after it has been sent to a function, that is `void printArray(int values[]);` or `void printArray(int *values);`, sends a pointer to the first element of the array when called as follows: `printArray(myArray);`. Therefore, within the function, the above calculation is the size of a pointer divided by the size of an int and not of the array.

- Iterating through the array can take place either using the index or using pointer arithmetic

```
void printArray(int *values, int size) {
    for (int i = 0; i < size; i++) {
        cout << values[i] << " ";
    }
    cout << endl;
}
```

or

```
void printArray(int values[], int size) {
    for (int i = 0; i < size; i++) {
        cout << *(values + i) << " ";
    }
    cout << endl;
}
```

- An array, where every element of a 1D array is an array is referred to as a matrix or a two-dimensional (2D) array.
- For example, the votes that candidates achieved in different voting areas is an example of data that can be stored as a matrix

	Candidate			
Area	A	B	C	D
1	10	7.5	1	0.5
2	22	15	0.7	0
3	6.5	8	5	4

- A matrix is defined by its number of rows and then its number of columns. The matrix above has 3 rows and 4 columns, and contains floating point values. It can be defined by:

```
float  matrix [3][4];
```

- Assigning values to the matrix can either be a set of default values or entered from the keyboard.
- Values to be assigned to the matrix row-by-row using the indexes of the row (`i`) and column (`j`), where `0` $\leq$ `i` $<$ `3` and `0` $\leq$ `j` $<$ `4`, that is: `matrix[i][j]`.
- Require a nested loop to initialise the values of the matrix to `0.0`.

```
for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 4; j++) {
                matrix[i][j] = 0.0;
        }
}
```

- Alternative initialisation of the matrix:

```
float matrix[3][4] = {{0.0,0.0,0.0,0.0},
                      {0.0,0.0,0.0,0.0},
                      {0.0,0.0,0.0,0.0}};
```

As with 1D arrays, it is possible to calculate the size of a 2D array.

- We can calculate the number of columns in a row by:
  `int cols = sizeof(matrix[0])/ sizeof( float );`
- Calculating the number of rows requires us to know the number of columns as well as the size of a float:
  `int rows = sizeof(matrix)/ sizeof( float )/cols;`

Write a function that prints the matrix to the console. The function is called as follows from the main function:
`printMatrix(matrix,rows,cols);`

- Note, the number of columns must be specified as a constant when using the brackets notation for formal parameters.

```
void printMatrix(float m[][4], int r, int c) {
    for (int i = 0; i < r; i++) {
            for (int j = 0; j < c; j++) {
                    cout << fixed << setprecision(1)
                         << m[i][j] << " ";
            }
            cout << endl;
    }
}
```

  This means the function only works for 2D arrays holding 4 floats per row.

- The number of columns must be specified when accepting a static array, even when the formal parameter is defined as a pointer such as:

```
void printMatrix(float (*m)[4], int r, int c);
```

- This is a major limitation.

- Write a function to read values from the keyboard into the matrix.

```
// void readMatrix(float m[][4], int r, int c) {    // OR
void readMatrix(float (*m)[4], int r, int c) {
    for (int i = 0; i < r; i++) {
            for (int j = 0; j < c; j++) {
                cin >> m[i][j];
            }
            cout << endl;
    }
}
```

- As with 1D arrays, 2D arrays can be stepped though using pointer arithmetic instead of indices.
- Consider the following code showing the `printMatrix` function using pointer arithmetic.

```
void printMatrixPtrArith(float m[][4], int r, int c) {
    for (int i = 0; i < r; i++) {
            for (int j = 0; j < c; j++) {
                    cout << fixed << setprecision(1)
                         << *(*(m+i)+j) << '\t';
            }
            cout << endl;
    }
}
```

- Note how the row increment, `*(m+i)`, interacts with the column increment, `*(*(m+i)+j)`, where `i` is the row increment and `j` the column increment.

Exercise

Write a library to manipulate matrices that are no larger than `4 x 4`.

- Add the `printMatrix` and `readMatrix` function definitions in a header file, `Matrix.h` and their corresponding implementations in the `Matrix.cpp` implementation file.
- Write a matrix addition function, `void addMatrices(float a[][4], float b[][4], float result[][4], int rows, int cols);`. Matrix addition is defined as follows:
  For two matrices A and B of the same dimensions $m \times n$:

$$C_{ij} = A_{ij} + B_{ij}, 0 \leq i < m, 0 \leq j < n$$

  where C is the resulting matrix.

Homework exercise

- Write a scalar matrix multiplication function, `void scalarMultiplyMatrix(float a[][4], float result[][4], int rows, int cols, float scalar);`. Scalar matrix multiplication is defined as follows:
  For a matrix A of dimension $m \times n$ and a scalar k:

$$B_{ij} = k \times A_{ij}, 0 \leq i < m, 0 \leq j < n$$

  where B is the resulting matrix.

- Write an element-wise division function, `void divideMatrices(float a[][4], float b[][4], float result[][4], int rows, int cols);`. For two matrices A and B of the same dimensions $m \times n$:

$$C_{ij} = \frac{A_{ij}}{B_{ij}}, 0 \leq i < m, 0 \leq j < n, B_{ij} \neq 0$$

  where C is the resulting matrix. Note: Division by zero is undefined.

Note:

- Our matrices may have less than 4 rows, but must have 4 columns. This is due to the requirement that the second array index must be 4.
- We cannot do matrix multiplication unless we pad our arrays and only use the part of the arrays where there is valid data. If you have time, experiment with this and write a matrix multiplication algorithm. The mathematical definition is given by:
  For matrices A of dimension $m \times n$ and B of dimension $n \times p$:

$$C_{ij} = \sum_{k=0}^{n-1} A_{ik} \times B_{kj}, 0 \leq i < m, 0 \leq j < p$$

  where C is the resulting matrix of dimension $m \times p$.
- How do we solve these limitations of these implementations in order to write a more generic Matrix library?
- Read up about dynamic arrays.